SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING) Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA06	**Course Title:**	Design and Analysis of Algorithms
CO Assessed:	CO2 – Manipulation (BL1, BL2, BL3)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	40% of CIA	**Total Marks:**	100
CO2 Statement:	*Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems (BL1, BL2, BL3)*		
Student Name:		**Reg. No.:**	
Section / Batch:		**Date:**	

Instructions to Students

- Answer the following question. Total Marks: 100.
- Analyze each problem carefully before applying the appropriate Brute Force technique.
- Clearly show all intermediate steps, comparisons, iterations, and logical reasoning used in solving the problems. • Apply suitable algorithms for sorting, searching, Closest-Pair, and Convex-Hull problems wherever required. • Justify the correctness and efficiency of the proposed solution using appropriate complexity analysis. • Use diagrams, tables, or stepwise illustrations wherever necessary for better explanation.
- Maintain neat presentation, proper algorithmic notation, and structured analytical reasoning throughout the answers.

Question Type	Focus Area	Questions
Application-Based Questions	Apply Brute Force techniques for sorting, searching, Closest-Pair and Convex-Hull problem analysis	Q1

SECTION A – APPLICATION BASED QUESTIONS

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

RUBRICS FOR CALCULATION

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning

Understanding of Problem Context	20%	Clearly understands the problem and accurately identifies all requirements	Good understanding with minor gaps	Basic understanding; some requirements unclear	Poor understanding or misinterpretation of the problem
Application of Concepts	30%	Accurately applies appropriate concepts to solve complex problems	Applies relevant concepts correctly with minor errors	Applies basic concepts with limited accuracy	Fails to apply appropriate concepts
Problem-Solving Approach	20%	Logical, well-structured, and efficient approach	Mostly logical approach with minor gaps	Basic approach with some inconsistencies	Illogical or unclear approach
Accuracy of Solution	20%	Completely correct solution with proper justification	Mostly correct with minor errors	Partially correct solution	Incorrect or incomplete solution
Clarity	10%	Clear, well-organized, and properly explained solution	Understandable with minor clarity issues	Limited clarity and explanation	Poorly presented and difficult to understand

Marks Evaluation:

Criteria	Wt.	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Problem Context	20%				
Application of Concepts	30%				
Problem-Solving Approach	20%				
Accuracy of Solution	20%				
Clarity	10%				
Total Marks (100):					

Faculty In-charge	Course Coordinator	Head of Department
Name:	Name:	Name:
Signature: _____	Signature: _____	Signature: _____
Date: _____	Date: _____	Date: _____

ASSESSMENT

CO2 - AT2

Name : Ch. Bhavya Sri

Regno : 192411249

Couse Code : CSA0612

Couse : Design and analysis algorithms

Q 23) Selection Sort – Repeated Minimum elements

1. Problem Interpretation

The given dataset is:

$$A = [12, 5, 7, 5, 18, 9]$$

The objective is to sort the given array in ascending order using the selection Sort algorithm, which is a Brute force sorting technique. The dataset contains duplicate minimum elements (5 and 5). We must observe how selection Sort indicates repeated minimum values, display the array after every pass, Count the total of Comparisons and swaps, analyze the time Comple-xity, verify the final sorted result, and it interpret the behavior of the algorithm.

Why Selection Sort?

- It is simple to understand and implement.
- It works Correctly even when duplicate values are present.
- It always performs the same number of Comparisons regardless of the input order.
- It requires only a small amount of extra memory (in-place Sorting)

Selection Sort Algorithm

Step 1: Start from the first element

Step 2: Find the minimum element in the unsorted portion.

Step 3: Swap the minimum element with the first unsorted position.

Step 4: Move to the next position.

Step 5: Repeat until the entire array becomes sorted.

Step-by-step Sorting Process

Initial Array - [12, 5, 7, 5, 18, 9]

Pass 1 Current array: [12, 5, 7, 5, 18, 9]

Minimum element in the array = 5 (first occurence)

Swap 12 and 5

Result: [5, 12, 7, 5, 18, 9]

Comparisons = 5, Swaps = 1

Pass 2 Unsorted part: [12, 7, 5, 18, 9]

Minimum element = 5

Swap 12 and 5

Result: [5, 5, 7, 12, 18, 9]

Comparisons = 4, Swaps = 1

Pass 3

Unsorted part:

[7, 12, 18, 9]

Minimum element = 7,

Already in Correct position

Result: [5, 5, 7, 12, 18, 9]

Comparisons = 3, Swaps = 0

Pass 4 Unsorted part: [12, 18, 9]

Minimum element = 9

Swap 12 and 9

Result:

[5, 5, 7, 9, 18, 12]

Comparisons = 2, Swaps = 1

Pass 5

Unsorted part: [18, 12]

Minimum element = 12

Swap 18 and 12

Final array: [5, 5, 7, 9, 12, 18]

Comparisons = 1

Swaps = 1

Total Comparisons: 5+4+3+2+1 = 15

Time Complexity Analysis

Best Case:

Even if the array is already sorted, of Selection Sort Compares every element.

Time Complexity = $O(n^2)$

Average Case

Average Comparisons remains the same

Time Complexity = $O(n^2)$

Worst Case:

When the array is in reverse order, Selection Sort still performs the same Comparisons.

Time Complexity = $O(n^2)$

Space Complexity = $O(1)$

Verification of the final Result

Final sorted array: [5, 5, 7, 9, 12, 18]

Checking the order:

$5 \leq 5 \leq 7 \leq 9 \leq 12 \leq 18$

Hence, the array is Correctly Stored in ascending order.